

Lab - Azure Blobs - Pessimistic Concurrency

To Begin with the Lab

Summary

This lab demonstrates **pessimistic concurrency** in Azure Blob Storage using **Blob Leases**. A lease locks a blob for a specified duration, preventing other clients from modifying or deleting it. The application acquires a lease, updates the blob using the lease ID, and then releases the lease after the operation is complete.

Understanding

Pessimistic concurrency is a technique used when you want to **prevent multiple users or applications from modifying the same blob at the same time**. Instead of allowing concurrent updates and checking for conflicts later (optimistic concurrency), Azure Blob Storage places a **lease (lock)** on the blob.

A lease grants exclusive write access to the client that acquired it. Any other client attempting to update or delete the blob without the correct lease ID receives an error. The lease can be acquired for a fixed duration (15–60 seconds) or indefinitely until released.

To modify a leased blob, the application must include the **Lease ID** in the upload request. Once the update is complete, the lease should be released so other clients can access the blob.

Example:

Imagine a financial application updating a daily transaction report stored as a blob.

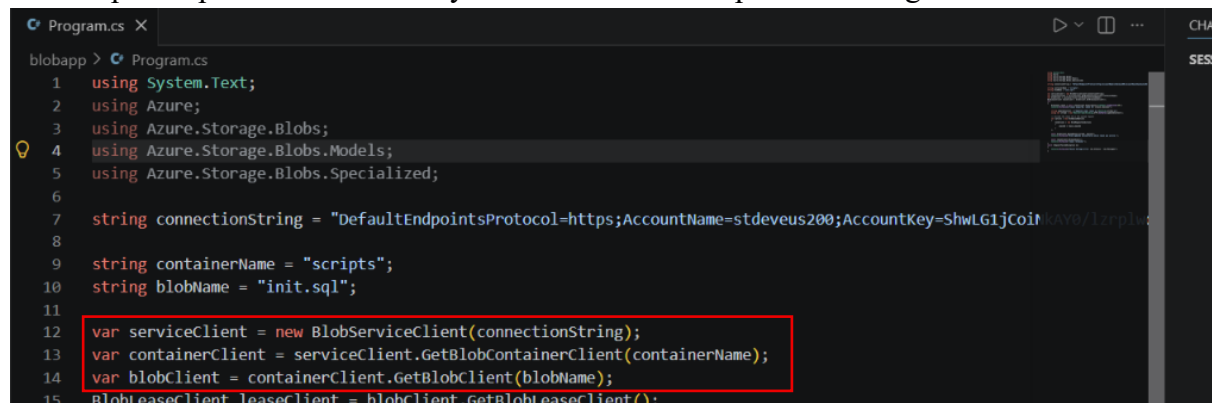
- Application A acquires a 60-second lease on the report.
- During this time, Application B cannot modify or delete the report.
- Application A updates the report while providing the lease ID.
- After the update, Application A releases the lease.
- Now other applications can safely modify the blob.

This ensures that only one application can make changes at a time, avoiding conflicting updates.

Lab Steps

Step 1: Create the Blob Client

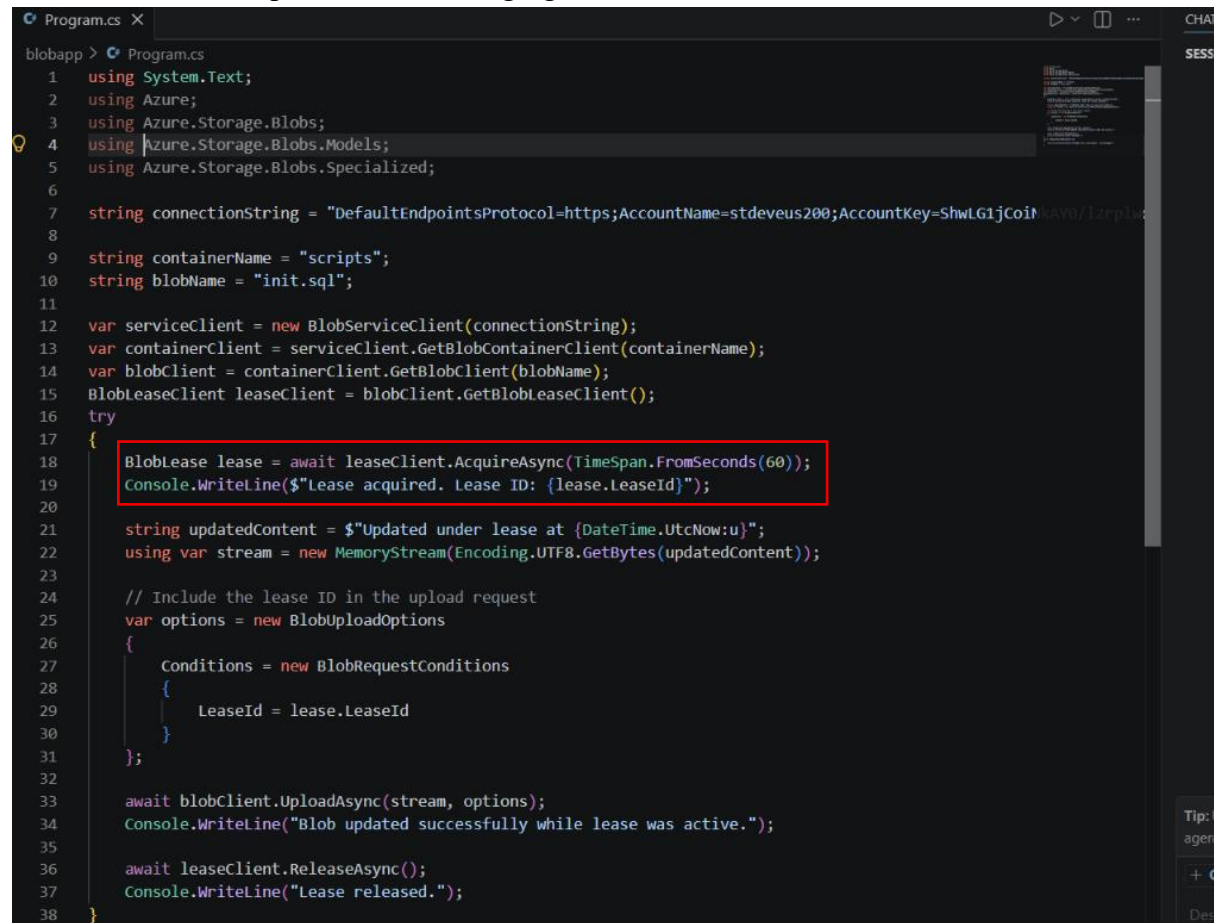
- Connect to the Azure Storage account.
- Create a BlobClient for the target blob.
- Wrap the operations inside a try-catch block for exception handling.



```
Program.cs
blobapp > Program.cs
1 using System.Text;
2 using Azure;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.Models;
5 using Azure.Storage.Blobs.Specialized;
6
7 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdeveus200;AccountKey=ShwLG1jcoiNkAv0/1zplz...";
8
9 string containerName = "scripts";
10 string blobName = "init.sql";
11
12 var serviceClient = new BlobServiceClient(connectionString);
13 var containerClient = serviceClient.GetBlobContainerClient(containerName);
14 var blobClient = containerClient.GetBlobClient(blobName);
15 BlobLeaseClient leaseClient = blobClient.GetBlobLeaseClient();
```

Step 2: Get the Blob Lease Client

- Create a BlobLeaseClient from the existing BlobClient.
- This client is responsible for managing blob leases.



```
Program.cs X
blobapp > Program.cs
1 using System.Text;
2 using Azure;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.Models;
5 using Azure.Storage.Blobs.Specialized;
6
7 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdevs200;AccountKey=ShwLG1jCoimAY8/1zrplx...";
8
9 string containerName = "scripts";
10 string blobName = "init.sql";
11
12 var serviceClient = new BlobServiceClient(connectionString);
13 var containerClient = serviceClient.GetBlobContainerClient(containerName);
14 var blobClient = containerClient.GetBlobClient(blobName);
15 BlobLeaseClient leaseClient = blobClient.GetBlobLeaseClient();
16 try
17 {
18     BlobLease lease = await leaseClient.AcquireAsync(TimeSpan.FromSeconds(60));
19     Console.WriteLine($"Lease acquired. Lease ID: {lease.LeaseId}");
20
21     string updatedContent = $"Updated under lease at {DateTime.UtcNow:u}";
22     using var stream = new MemoryStream(Encoding.UTF8.GetBytes(updatedContent));
23
24     // Include the lease ID in the upload request
25     var options = new BlobUploadOptions
26     {
27         Conditions = new BlobRequestConditions
28         {
29             LeaseId = lease.LeaseId
30         }
31     };
32
33     await blobClient.UploadAsync(stream, options);
34     Console.WriteLine("Blob updated successfully while lease was active.");
35
36     await leaseClient.ReleaseAsync();
37     Console.WriteLine("Lease released.");
38 }
```

Step 3: Acquire the Lease

- Acquire a lease on the blob.
- Specify a lease duration (for example, 60 seconds).
- Store the returned Lease ID.
- Display the Lease ID in the console.

Result:

- The blob is locked for the specified duration.
- Other clients cannot modify or delete the blob without the Lease ID.

```
Program.cs X
blobapp > Program.cs
1 using System.Text;
2 using Azure;
3 using Azure.Storage.Blobs;
4 using Azure.Storage.Blobs.Models;
5 using Azure.Storage.Blobs.Specialized;
6
7 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdevs200;AccountKey=ShwLG1jCoitAY8/lzrplk...";
8
9 string containerName = "scripts";
10 string blobName = "init.sql";
11
12 var serviceClient = new BlobServiceClient(connectionString);
13 var containerClient = serviceClient.GetBlobContainerClient(containerName);
14 var blobClient = containerClient.GetBlobClient(blobName);
15 BlobLeaseClient leaseClient = blobClient.GetBlobLeaseClient();
16 try
17 {
18     BlobLease lease = await leaseClient.AcquireAsync(TimeSpan.FromSeconds(60));
19     Console.WriteLine($"Lease acquired. Lease ID: {lease.LeaseId}");
20
21     string updatedContent = $"Updated under lease at {DateTime.UtcNow:u}";
22     using var stream = new MemoryStream(Encoding.UTF8.GetBytes(updatedContent));
23
24     // Include the lease ID in the upload request
25     var options = new BlobUploadOptions
26     {
27         Conditions = new BlobRequestConditions
28         {
29             LeaseId = lease.LeaseId
30         }
31     };
32
33     await blobClient.UploadAsync(stream, options);
34     Console.WriteLine("Blob updated successfully while lease was active.");
35
36     await leaseClient.ReleaseAsync();
37     Console.WriteLine("Lease released.");
38 }
```

Step 4: Update the Blob

- Prepare the updated blob content.
- Create BlobUploadOptions.
- Set the Lease ID in the upload conditions.
- Upload the blob using the lease.

Important:

- Every write operation must include the Lease ID while the lease is active.
- Without the Lease ID, Azure rejects the update.

```
await blobClient.UploadAsync(stream, options);
Console.WriteLine("Blob updated successfully while lease was active.");

await leaseClient.ReleaseAsync();
Console.WriteLine("Lease released.");
}
catch (RequestFailedException ex)
{
    Console.WriteLine($"Azure Storage error: {ex.Status} - {ex.Message}");
}
```

Step 5: Release the Lease

- After the upload completes successfully, release the lease.
- The blob becomes available for other clients.

Result:

- Other applications can now update or delete the blob.

```

    await blobClient.UploadAsync(stream, options);
    Console.WriteLine("Blob updated successfully while lease was active.");

    await leaseClient.ReleaseAsync();
    Console.WriteLine("Lease released.");
}
catch (RequestFailedException ex)
{
    Console.WriteLine($"Azure Storage error: {ex.Status} - {ex.Message}");
}

```

Step 6: Build and Run the Application

- Add the required Azure Storage namespace if missing.
- Build the project using .NET build.

```

powershell blobapp X
PS D:\Dev3\09. Lab - Azure Blobs - Pessimistic concurrency/blobapp> dotnet build
Restore complete (2.4s)
  blobapp net10.0 succeeded (3.9s) -> bin\Debug\net10.0/blobapp.dll

Build succeeded in 6.9s
PS D:\Dev3\09. Lab - Azure Blobs - Pessimistic concurrency/blobapp>

```

- Run the application.
- Verify that the lease is acquired.
- Confirm that the Lease ID is displayed.
- Verify that the blob is updated successfully.
- Confirm that the lease is released.

```

powershell blobapp X
PS D:\Dev3\09. Lab - Azure Blobs - Pessimistic concurrency/blobapp> dotnet build
Restore complete (2.4s)
  blobapp net10.0 succeeded (3.9s) -> bin\Debug\net10.0/blobapp.dll

Build succeeded in 6.9s
PS D:\Dev3\09. Lab - Azure Blobs - Pessimistic concurrency/blobapp> dotnet run
Lease acquired. Lease ID: e5cc0396-e45f-476b-bd44-ef5cfe07eb90
Blob updated successfully while lease was active.
Lease released.
PS D:\Dev3\09. Lab - Azure Blobs - Pessimistic concurrency/blobapp>

```